ShanghaiTech University

CS101 Algorithms and Data Structures
Fall 2024

Homework 1

Wenye Xiong 2023533141

Due date: October 9, 2024, at 23:59

1. Please write your solutions in English.
2. Submit your solutions to Gradescope.
3. Set your FULL name to your Chinese name and your STUDENT ID correctly in Gradescope account settings.
4. If you want to submit a handwritten version, scan it clearly. `CamScanner` is recommended.
5. We recommend you to write in LaTeX.
6. When submitting, match your solutions to the problems correctly.
7. No late submission will be accepted.
8. Violations to any of the above may result in zero points.

1. (7 points) Academic Integrity

Please determine who violated academic integrity or committed plagiarism in the following situations. Tick ($\surd$) for the correct answers.

(a) (1') Alice posted one of the homework questions on Stack Overflow and copied the answer from others to her homework with some trivial modifications.

$\surd$ **Alice** ◯ Bob ◯ Both Alice and Bob ◯ Neither Alice nor Bob

(b) (1') Alice entered the question description into ChatGPT, then slightly modified ChatGPT's response and submitted it as her own answer.

$\surd$ **Alice** ◯ Bob ◯ Both Alice and Bob ◯ Neither Alice nor Bob

(c) (1') Bob lent Alice his laptop to play Genshin. Alice found Bob's solution to Homework 1 on the desktop and copied it using USB drive sneakily.

◯ Alice ◯ Bob $\surd$ **Both Alice and Bob** ◯ Neither Alice nor Bob

(d) (1') Alice and Bob are good friends. Alice asked if Bob could send her some code snippets so that she could have some ideas of what is going on in this programming assignment (promising, of course, not copying). Bob agreed and shared his code repository with Alice. Alice copied Bob's code, changed some variable names, and submitted the code to CS101 Online Judge.

◯ Alice ◯ Bob $\surd$ **Both Alice and Bob** ◯ Neither Alice nor Bob

(e) (1') To boost efficiency and save precious time, Bob used Copilot as an assistant to write code in his programming assignments.

◯ Alice $\surd$ **Bob** ◯ Both Alice and Bob ◯ Neither Alice nor Bob

(f) (1') Bob is Alice's boyfriend. In order to enhance their romantic relationship, Alice and Bob studied together in ShanghaiTech Library and collaborated on one homework question. They discussed about some algorithm using a whiteboard, without giving any exact solution.

◯ Alice ◯ Bob ◯ Both Alice and Bob $\surd$ **Neither Alice nor Bob**

(g) (1') Alice completed the programming assignment herself on Bob's computer and accidentally submitted her code to CS101 Online Judge using Bob's account. After that, Alice and Bob resubmitted their assignments using their own accounts respectively.

◯ Alice ◯ Bob $\surd$ **Both Alice and Bob** ◯ Neither Alice nor Bob

2. (12 points) Multiple Choices

Each question has **one or more** correct answer(s). Select all the correct answer(s). For each question, you will get 0 points if you select one or more wrong answers, but you will get 1 point if you select a non-empty subset of the correct answers.

Write your answers in the following table.

(a)	(b)	(c)	(d)
ADE	D	AD	A

(a) (3') Which of the following statements about arrays and linked lists are true?

A. A doubly linked list consumes more memory than a (singly) linked list of the same length.

B. Inserting an element into the middle of an array takes constant time.

C. Reversing a singly linked list takes constant time.

D. Given a pointer to some node in a doubly linked list, we are able to gain access to every node of it.

E. If we implement a queue using the circular array, the minimal memory we need is related to the maximal possible numbers of elements in the queue.

(b) (3') Suppose we use a circular array with an index range from 0 to $N-1$ to implement a queue, and currently Front is pointing at index m. We will know that if this queue is full with N elements, the last element in it should be stored at index = _____. (Options below are mapped to **Integers Modulo N**: $\mathbb{Z}_N = \{0, 1, 2, ..., N-1\}$)

A. 0

B. m

C. $N-1$

D. $m-1$

(c) (3') Which of the following statements is(are) correct?

A. $n! = o(n^n)$**.**

B. $n^n = O(n!)$

C. $(\log n)^2 = \omega(\sqrt{n})$.

D. $n + \log n = \Omega(n + \log\log n)$**.**

E. $n^{\log(n^2)} = O(n^2 \log(n^2))$.

(d) (3') Your two magic algorithms run in

$$f(n) = n\lceil\sqrt{n}\rceil(1 + (-1)^n) + 1$$

$$g(n) = n\lfloor\log n\rfloor$$

time, where n is the input size. Which of the following statements is true?

A. $f(n) = o\left(n^2\right)$**.**

B. $f(n) = \Omega\left(n\right)$.

C. $f(n) + g(n) = \Theta\left(n^{1.5}\right)$.

D. $f(n) + g(n) = \omega\left(n \log\left(1.5n\right)\right)$.

Here is the definition of Landau Symbols without using the limit, which may be helpful for you to strictly prove whether each choice is correct:

$$f(n) = \Theta(g(n)) : \exists c_1, c_2 \in \mathbb{R}^+, \exists n_0, \forall n > n_0, 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n).$$
$$f(n) = O(g(n)) : \exists c \in \mathbb{R}^+, \exists n_0, \forall n > n_0, 0 \leq f(n) \leq c \cdot g(n).$$
$$f(n) = \Omega(g(n)) : \exists c \in \mathbb{R}^+, \exists n_0, \forall n > n_0, 0 \leq g(n) \leq c \cdot f(n).$$
$$f(n) = o(g(n)) : \forall c \in \mathbb{R}^+, \exists n_0, \forall n > n_0, 0 \leq f(n) < c \cdot g(n).$$
$$f(n) = \omega(g(n)) : \forall c \in \mathbb{R}^+, \exists n_0, \forall n > n_0, 0 \leq g(n) < c \cdot f(n).$$

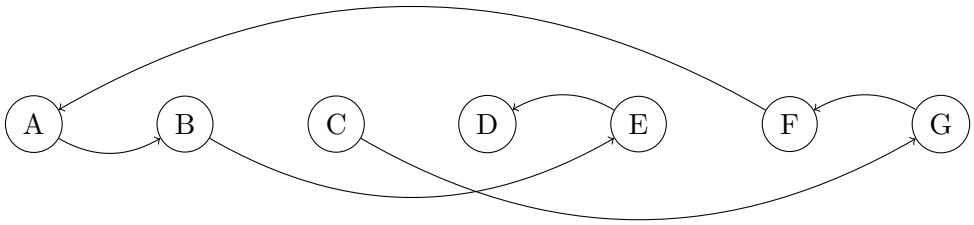
3. (10 points) Array Representation of Linked-List

Recall that we store a linked-list in an array to avoid memory allocation problems in the lecture. In this question, we use two arrays: `next` and `value` to implement a singly-linked-list.

The elements at each index in `next` and `value` are together to represent a node in the linked-list, where `next` represents the array index where the next node is located (-1 for already reaching the tail), and `value` represents the data stoblue in the node.

(a) (5') **Store a Linked-List in an Array**

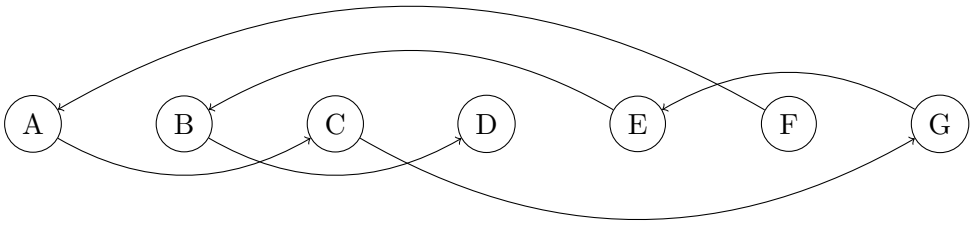
Fill in the table below to finish the array representation of the linked-list shown below. Fill in -1 to represent the end of the linked-list.



index	0	1	2	3	4	5	6
`value`	A	B	C	D	E	F	G
`next`	1	4	6	-1	3	0	5

(b) (5') **From Array to Linked-List**

Here are the arrays `next` and `value`. Please draw the linked-list below represented by these two arrays.



index	0	1	2	3	4	5	6
`value`	A	B	C	D	E	F	G
`next`	2	3	6	-1	1	0	4

4. (10 points) Vocabulary List

Alice is memorizing English words to enlarge her vocabulary in preparation for her approaching CET-4 test and she needs your help! Alice uses a `Node` to define a word and a linked list which contains the words that she is trying to memorize now. The structure of `Node` is as follows.

```
struct Node {
  string word;
  Node *next;
  Node(const string& _word, Node *_next)
      : word(_word), next(_next) {}
};
Node *head;
```

Each time she reviews her vocabulary list, she steps through the linked list starting from `head`. However, sometimes there is a word that she doesn't know, so she wants to grab the corresponding `Node` out and put it at the start of the linked list, so next time she can review it first.

For example, if the vocabulary list is

`algorithm` → `breath` → `capacity` → `degree`

and she doesn't know the word `capacity`, then she wants the list be modified to

`capacity` → `algorithm` → `breath` → `degree`

(a) (6') Alice needs your help to finish her function `move_to_first(Node *head, Node *p)` where `head` represents the first element of her non-empty vocabulary list and she wants to move the **next** word of `p` to the first. The function returns the **new** head pointer after this process. She can guarantee that `p` points to a `Node` in the linked list, but not the last one.

```
Node* move_to_first(Node *head, Node *p) {
    Node *to_move = p->next;
    // Write something below
    // ...
    // Write something above
}
```

Please help her complete the function.

Solution:

```
p->next = to_move->next;
to_move->next = head;
```

```
            head = to_move;
            return head;
```

(b) (4') Now Alice wants to reverse the entire list using the `move_to_first` function. Please help her complete the reversion function `reverse_list(Node *head)` where `head` is the head pointer of the vocabulary list to be reversed. The function returns the **new** head pointer after this process. She can guarantee that the list is non-empty.

```
Node* reverse_list(Node *head){
    Node *p = head;
    while(/* (1) */){
        /* (2) */ = move_to_first(/* (3) */, /* (4) */);
    }
    return head;
}
```

Please help her complete this function by filling your code into blank (1) to (4). Each blank should be just an expression or a variable.

Solution:

```
(1): p->next
(2): head
(3): head
(4): p
```

5. (10 points) Postfix Expression

Reverse Polish notation (RPN) is a mathematical notation in which operators follow their operands. Using a stack, we can evaluate postfix notation equations easily.

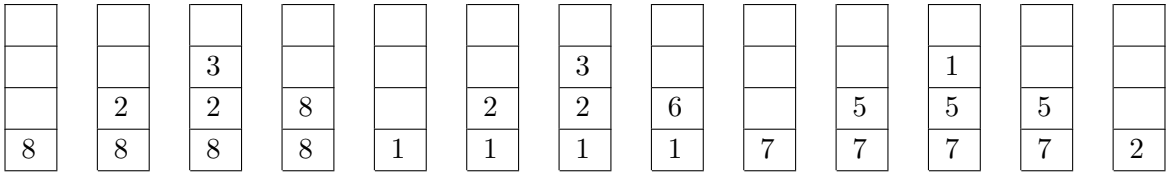
A postfix expression (Reverse-Polish Notation) with single-digit operands is shown below: (where ^ is the exponentiation operator)

`8 2 3 ^ / 2 3 * + 5 1 * -`

(a) (1') What's the in-fix expression of the expression above?

Solution: $(8/2^3) + (2 * 3) - (5 * 1)$

(b) (4') The changing of the stack to calculate the final result is:



(c) (3') Try to convert the following in-fix expression into post-fix expression: (You don't need to calculate them)

(1) $1 * 2 + 3$

Solution: 1 2 * 3 +

(2) $1 + 2 * 3 + (4 * 5 + 6) * 7$

Solution: 1 2 3 * + 4 5 * 6 + 7 * +

(3) $1 + (2 * 3\hat{}4)/(5 + 6) + 7$

Solution: 1 2 3 4 ^ * + 5 6 + / + 7 +

(d) (2') Please judge whether the following post-fix expression is legal. Tick ($\surd$) for the legal expressions.

◯ 1 2 * + 3 5 + $\surd$ 4 5 6 / * 1 / ◯ 1 + 2 - 3 + 4 $\surd$ 7 8 9 1 + - *

6. (11 points) Compare and Prove

For each pair of functions $f(n)$ and $g(n)$, give your answers whether $f(n) = o(g(n))$, $f(n) = \omega(g(n))$ or $f(n) = \Theta(g(n))$. Give a **proof** of your answer.

Note 1: Give your answer in the most precise form.

Note 2: You'd better try to prove them by calculating limits.

(a) (3') $f(n) = 1.01^n$ and $g(n) = n^{10}$

Solution: $f(n) = \omega(g(n))$
Proof:
According to the definition of ω, we need to prove that $\lim_{n\to\infty} \frac{1.01^n}{n^{10}} = \infty$.
And with the help of L'Hopital's Rule, we can get:
$\lim_{n\to\infty} \frac{1.01^n}{n^{10}} = \lim_{n\to\infty} \frac{1.01^n \ln(1.01)}{10n^9}$
$= \lim_{n\to\infty} \frac{1.01^n \ln^2(1.01)}{90n^8} = \cdots = \lim_{n\to\infty} \frac{1.01^n \ln^{10}(1.01)}{10!} = \infty$
So, $f(n) = \omega(g(n))$.

(b) (4') $f(n) = \log(n!)$ and $g(n) = \log(n^n)$

Solution: $f(n) = \Theta(g(n))$
Proof:
According to the definition of Θ, we need to prove that $\lim_{n\to\infty} \frac{\log(n!)}{\log(n^n)} = c, 0 < c < \infty$.
And with the help of Stirling's Formula, we can get:
$\lim_{n\to\infty} \frac{\log(n!)}{\log(n^n)} = \lim_{n\to\infty} \frac{\log(\sqrt{2\pi n}(\frac{n}{e})^n)}{\log(n^n)}$
$= \lim_{n\to\infty} \frac{\log(\sqrt{2\pi n})+\log((\frac{n}{e})^n)}{n\log n} = \lim_{n\to\infty} \frac{\log(\sqrt{2\pi n})+n\log(\frac{n}{e})}{n\log n}$
$= \lim_{n\to\infty} \frac{\log(\sqrt{2\pi n})}{n\log n} + \lim_{n\to\infty} \frac{n\log(\frac{n}{e})}{n\log n} = 0 + 1 = 1$
So, $f(n) = \Theta(g(n))$.

(c) (4') $f(n) = n^{1+\varepsilon}, \varepsilon \in \mathbb{R}^+$ and $g(n) = n(\log n)^k, k \in \mathbb{Z}^+$

Solution: $f(n) = \omega(g(n))$
Proof:
According to the definition of ω, we need to prove that $\lim_{n\to\infty} \frac{n^{1+\varepsilon}}{n(\log n)^k} = \infty$.
$\lim_{n\to\infty} \frac{n^{1+\varepsilon}}{n(\log n)^k} = \lim_{n\to\infty} \frac{n^{\varepsilon}}{(\log n)^k}$
Next, we can use L'Hopital's Rule to get:
$\lim_{n\to\infty} \frac{n^{\varepsilon}}{(\log n)^k} = \lim_{n\to\infty} \frac{\varepsilon n^{\varepsilon}}{k(\log n)^{k-1}} = \cdots = \lim_{n\to\infty} \frac{\varepsilon^k n^{\varepsilon}}{k!} = \infty$
So, $f(n) = \omega(g(n))$.

7. (11 points) Problem Analysis

Consider the following basic problem:

You're given an array A consisting of n integers $A[1], A[2], ..., A[n]$. You'd like to output a two-dimensional n-by-n array B in which $B[i, j]$ (for $i < j$) contains the sum of array entries $A[i]$ through $A[j]$ – that is, the sum $A[i] + A[i + 1] + ... + A[j]$. (the value of array entry $B[i, j]$ is left unspecified whenever $i \geq j$.)

Here's the pseudocode of a simple algorithm to solve this problem. (Initially $B[i, j] = 0$ for all valid i, j)

```
1: for i = 1 to n do
2:     for j = i + 1 to n do
3:         for k = i to j do
4:             B[i, j] ← B[i, j] + A[k]
5:         end for
6:     end for
7: end for
```

For the following questions,

- Define $T(n)$ as the total running time of the algorithm.
- Suppose each operation like $B[i, j] \leftarrow B[i, j] + A[k]$ costs a constant time c.
- Ignore the time that `for` loop iterations take.

(a) (1') Give out a function f satisfying that the running time of the algorithm is $\Theta(f(n))$. That is, $T(n) = \Theta($_____n^3_____$)$. (Give your answer in the most simplified form.)

(b) (3') For this function chosen f, show that the running time of the algorithm on an input of size n is upper bounded by $f(n)$. That is, prove $T(n) = O(f(n))$.

> **Solution:** We can see that the algorithm contains three nested loops. Since the outer for-loop goes n times from i=1 to n (O(n)), the middle for-loop goes up to n times from j=i+1 to n (actually average run time is n/2), and the inner for-loop goes up to n times from i to j, the total running time is up to n^3. So, we can safely say that $T(n) = O(n^3)$.

(c) (4') For the same function f, show that the running time of the algorithm on an input of size n is also lower bounded by $f(n)$. That is, prove $T(n) = \Omega(f(n))$.

> **Solution:** Consider the running times during the whole process when $i \leq \frac{n}{4}$ and $j \geq \frac{3n}{4}$. The running time of the inner loop is $j - i + 1 > \frac{n}{2}$. What's more, there are totally $\frac{n}{4}^2$ pairs of i and j satisfying the condition. So, the total running time is lower bounded by $\frac{n}{4}^2 \times \frac{n}{2} = \frac{n^3}{32}$. So, we can safely say that $T(n) = \Omega(n^3)$.

(d) (3') Although the algorithm you just analyzed is the most natural way to solve the problem—after all, it just iterates through the relevant entries of the array B, filling in a value for each—it contains some highly unnecessary sources of inefficiency. Give a different algorithm to solve the problem, with an asymptotically better running time. In other words, you should design another algorithm with running time $\Theta(g(n))$, where $\lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$.
Just write the pseudocode below.

Solution:

for $i = 1$ **to** n **do**
  $B[i, i] \leftarrow A[i]$
  for $j = i + 1$ **to** n **do**
    $B[i, j] \leftarrow B[i, j - 1] + A[j]$
  end for
end for